Internationalization in Java

Sophomore Level: A Localized Application

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-15

Outline

1. A Localized Application
2. How Bundles Are Found
3. Bundle Files and Encodings
4. MessageFormat in Depth
5. Reading Localized Input
6. Comparing Text
7. Testing in Many Locales
8. Summary

A Localized Application

The project

Terminal codeoutput/Build.txt

```
$ find src -type f | sort
src/edu/yeditepe/shop/i18n/Messages_de.properties
src/edu/yeditepe/shop/i18n/Messages.properties
src/edu/yeditepe/shop/i18n/Messages_tr.properties
src/edu/yeditepe/shop/Main.java
$ javac -d out src/edu/yeditepe/shop/Main.java
```

- Bundles are **resources**: they live in a package (`edu.yeditepe.shop.i18n`) next to the code.
- The **base name** is the full name: `edu.yeditepe.shop.i18n.Messages`.
- One file per language; the base file (no suffix) is English and has every key.

The bundles

codeoutput/Shop/src/edu/yeditepe/shop/i18n/Messages.properties **properties**

```
# base bundle (English): used when no better file exists
title=Order summary
items={0,choice,0#no items|1#one item|1<{0,number,integer} items}
total=Total: {0}
delivery=Delivery on {0}
```

codeoutput/Shop/src/edu/yeditepe/shop/i18n/Messages_tr.properties **properties**

```
# Turkish
title=Sipariş özeti
items={0,choice,0#ürün yok|1<{0,number,integer} ürün}
total=Toplam: {0}
delivery={0} tarihinde teslim
```

codeoutput/Shop/src/edu/yeditepe/shop/i18n/Messages_de.properties **properties**

```
# German
title=Bestellübersicht
items={0,choice,0#keine Artikel|1#ein Artikel|1<{0,number,integer} Artikel}
total=Summe: {0}
delivery=Lieferung am {0}
```

The code: no text, no format in it

codeoutput/Shop/src/edu/yeditepe/shop/Main.java — Java

```java
public class Main {
    public static void main(String[] args) {
        Locale l = args.length > 0 ? Locale.forLanguageTag(args[0]) : Locale.getDefault();
        ResourceBundle rb = ResourceBundle.getBundle("edu.yeditepe.shop.i18n.Messages", l);

        int count = 3;
        BigDecimal total = new BigDecimal("1249.90").multiply(BigDecimal.valueOf(count));
        NumberFormat money = NumberFormat.getCurrencyInstance(l);
        money.setCurrency(Currency.getInstance("TRY"));      // prices are in lira
        String day = LocalDate.of(2026, 10, 20)
            .format(DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).withLocale(l));

        System.out.println(rb.getString("title"));
        System.out.println(text(rb, l, "items", count));
        System.out.println(text(rb, l, "total", money.format(total)));
        System.out.println(text(rb, l, "delivery", day));
    }

    static String text(ResourceBundle rb, Locale l, String key, Object... values) {
        return new MessageFormat(rb.getString(key), l).format(values);
    }
}
```

Every text comes from the bundle; every number, amount and date is formatted for the locale `l`. The Turkish `items` has no `1#` case: *1 ürün*, *3 ürün*.

Building: the bundles must be copied

Terminal codeoutput/Build.txt

```
$ find src -type f | sort
src/edu/yeditepe/shop/i18n/Messages_de.properties
src/edu/yeditepe/shop/i18n/Messages.properties
src/edu/yeditepe/shop/i18n/Messages_tr.properties
src/edu/yeditepe/shop/Main.java
$ javac -d out src/edu/yeditepe/shop/Main.java
$ mkdir -p out/edu/yeditepe/shop/i18n
$ cp src/edu/yeditepe/shop/i18n/*.properties out/edu/yeditepe/shop/i18n/
$ find out -type f | sort
out/edu/yeditepe/shop/i18n/Messages_de.properties
out/edu/yeditepe/shop/i18n/Messages.properties
out/edu/yeditepe/shop/i18n/Messages_tr.properties
out/edu/yeditepe/shop/Main.class
$ java -cp out edu.yeditepe.shop.Main en-US
Order summary
3 items
Total: TRY3,749.70
Delivery on October 20, 2026
$ java -cp out edu.yeditepe.shop.Main tr-TR
Sipariş özeti
3 ürün
Toplam: ₺3.749,70
20 Ekim 2026 tarihinde teslim
```

`javac` only writes `.class` files. The `.properties` files must be copied into `out` (Maven and Gradle do this for `src/main/resources`). **Windows**: `mkdir` and `copy` with `\`.

In a JAR

Terminal codeoutput/Jar.txt

```
$ jar --create --file shop.jar --main-class edu.yeditepe.shop.Main -C out .
$ jar tf shop.jar | grep properties
edu/yeditepe/shop/i18n/Messages.properties
edu/yeditepe/shop/i18n/Messages_de.properties
edu/yeditepe/shop/i18n/Messages_tr.properties
$ java -jar shop.jar
Order summary
3 items
Total: TRY3,749.70
Delivery on October 20, 2026
$ LANG=tr_TR.UTF-8 java -jar shop.jar
Sipariş özeti
3 ürün
Toplam: ₺3.749,70
20 Ekim 2026 tarihinde teslim
$ java -Duser.language=de -Duser.country=AT -jar shop.jar
Bestellübersicht
3 Artikel
Summe: TRY 3.749,70
Lieferung am 20. Oktober 2026
```

The bundles travel inside the JAR. Without an argument the program uses the default locale: `de-AT` has no file, so it uses `Messages_de`; the money format is Austrian (`TRY 3.749,70`).

How Bundles Are Found

The search order

Request `de-CH`, default locale `tr-TR`:

1. `Messages_de_CH`
2. `Messages_de`
3. `Messages_tr_TR` (default locale!)
4. `Messages_tr`
5. `Messages`

The first file that **exists** is returned. Its *parents* are the later files of the same language and the base file, so missing keys are still found.

Only when nothing matches

Steps 3–4 are tried only if neither `_de_CH` nor `_de` exists. Then a French user on a Turkish server gets **Turkish**.

Each file can be

a `.properties` file, or a class `Messages_de extends ListResourceBundle` (for values that are not text, e.g. images or objects). Classes are tried first.

The default-locale trap

Terminal codeoutput/Trap.txt

```
$ java -cp out edu.yeditepe.shop.Main fr-FR
Order summary
3 items
Total: 3 749,70 TRY
Delivery on 20 octobre 2026
$ java -Duser.language=tr -Duser.country=TR -cp out edu.yeditepe.shop.Main fr-FR
Sipariş özeti
3 ürün
Toplam: 3 749,70 TRY
20 octobre 2026 tarihinde teslim
```

The same request `fr-FR`: on an English server the texts are English (base file); on a Turkish server they are **Turkish**, while money and date are French. Mixed languages on one screen.

Turning the fallback off

codeoutput/Lookup/Lookup.java **Java**

```java
import java.util.Locale;
import java.util.ResourceBundle;
import java.util.ResourceBundle.Control;

public class Lookup {
    public static void main(String[] args) {
        Locale want = Locale.forLanguageTag(args[0]);
        ResourceBundle a = ResourceBundle.getBundle("Texts", want);
        ResourceBundle b = ResourceBundle.getBundle("Texts", want,
            Control.getNoFallbackControl(Control.FORMAT_DEFAULT));   // no default locale
        System.out.println("default control: [" + a.getLocale() + "] " + a.getString("hi"));
        System.out.println("no fallback:     [" + b.getLocale() + "] " + b.getString("hi"));
    }
}
```

`Control.getNoFallbackControl` skips the default locale: unknown languages get the base file, whatever the server's locale is. Servers that serve many users should use it.

Turning the fallback off

codeoutput/Lookup/Lookup.java — Java

```java
import java.util.Locale;
import java.util.ResourceBundle;
import java.util.ResourceBundle.Control;

public class Lookup {
    public static void main(String[] args) {
        Locale want = Locale.forLanguageTag(args[0]);
        ResourceBundle a = ResourceBundle.getBundle("Texts", want);
        ResourceBundle b = ResourceBundle.getBundle("Texts", want,
            Control.getNoFallbackControl(Control.FORMAT_DEFAULT));   // no default locale
        System.out.println("default control: [" + a.getLocale() + "] " + a.getString("hi"));
        System.out.println("no fallback:     [" + b.getLocale() + "] " + b.getString("hi"));
    }
}
```

Terminal — codeoutput/Lookup.txt

```
$ ls
Lookup.java
Texts_de.properties
Texts.properties
Texts_tr.properties
$ java Lookup.java de-CH
default control: [de] Hallo
no fallback:     [de] Hallo
$ java Lookup.java fr
default control: [] Hello
no fallback:     [] Hello
$ java -Duser.language=tr Lookup.java fr
default control: [tr] Merhaba
no fallback:     [] Hello
```

`Control.getNoFallbackControl` skips the default locale: unknown languages get the base file, whatever the server's locale is. Servers that serve many users should use it.

Bundle Files and Encodings

Encoding of .properties files

Terminal codeoutput/Encoding.txt

```
$ file *.properties
Escaped_tr.properties: ASCII text
Texts_tr.properties:   ISO-8859 text
Utf8_tr.properties:    Unicode text, UTF-8 text
$ java Show.java Texts
Texts: Çýkýþ
$ java Show.java Escaped
Escaped: Çıkış
$ java Show.java Utf8
Utf8: Çıkış
$ iconv -f ISO-8859-9 -t UTF-8 Texts_tr.properties > tmp && mv tmp Texts_tr.properties
$ java Show.java Texts
Texts: Çıkış
```

- Since Java 9 bundles are read as **UTF-8**. If a file is not valid UTF-8, Java reads it as ISO-8859-1: a Turkish ISO-8859-9 file shows Çýkýþ.
- \u0131 escapes work everywhere (the only way before Java 9).
- Save bundle files as UTF-8 in the editor (**Windows**: Notepad *Save as, Encoding: UTF-8*).

MessageFormat in Depth

Pitfalls of MessageFormat

codeoutput/Pitfalls.java — Java

```java
import java.text.MessageFormat;
import java.util.Locale;

public class Pitfalls {
    static String f(String pattern, Object... v) {
        return new MessageFormat(pattern, Locale.US).format(v);
    }
    public static void main(String[] args) {
        System.out.println(f("Founded in {0}", 2026));              // a number!
        System.out.println(f("Founded in {0,number,#}", 2026));
        System.out.println(f("Founded in {0}", "2026"));            // or a String
        System.out.println(f("It's {0}", "late"));                  // ' starts a quote
        System.out.println(f("It''s {0}", "late"));
        System.out.println(f("Use '{0}' for the first value", "x")); // quoted: no value
        System.out.println(f("{0,number,percent} done", 0.75));
        System.out.println(f("{1} before {0}", "A", "B"));           // any order
    }
}
```

A number argument is formatted with grouping: `2,026`. A single `'` quotes text, so `It's` loses its apostrophe and `'{0}'` is printed literally. Translators must know these rules.

Pitfalls of MessageFormat

codeoutput/Pitfalls.java — Java

```java
import java.text.MessageFormat;
import java.util.Locale;

public class Pitfalls {
    static String f(String pattern, Object... v) {
        return new MessageFormat(pattern, Locale.US).format(v);
    }
    public static void main(String[] args) {
        System.out.println(f("Founded in {0}", 2026));              // a number!
        System.out.println(f("Founded in {0,number,#}", 2026));
        System.out.println(f("Founded in {0}", "2026"));            // or a String
        System.out.println(f("It's {0}", "late"));                  // ' starts a
        quote
        System.out.println(f("It''s {0}", "late"));
        System.out.println(f("Use '{0}' for the first value", "x")); // quoted: no
        value
        System.out.println(f("{0,number,percent} done", 0.75));
        System.out.println(f("{1} before {0}", "A", "B"));           // any order
    }
}
```

Terminal — codeoutput/Pitfalls.txt

```
Founded in 2,026
Founded in 2026
Founded in 2026
Its {0}
It's late
Use {0} for the first value
75% done
B before A
```

A number argument is formatted with grouping: `2,026`. A single `'` quotes text, so `It's` loses its apostrophe and `'{0}'` is printed literally. Translators must know these rules.

Argument types

pattern	meaning
`{0}`	`toString()`, numbers and dates localized
`{0,number}`, `{0,number,integer}`, `{0,number,percent}`	numbers
`{0,number,#.##}`	a `DecimalFormat` pattern
`{0,date,long}`, `{0,time,short}`	`java.util.Date` only, not `java.time`
`{0,choice,0#none|1#one|1<{0} many}`	select a text by a number
`''`	one apostrophe
`'{'`	a literal brace

Format `java.time` values with `DateTimeFormatter` first and pass the `String`, as `Main` does.

Reading Localized Input

Parsing numbers and dates safely

codeoutput/ParseInput.java — Java

```
10      static void number(String s, Locale l) {
11          ParsePosition pos = new ParsePosition(0);
12          Number n = NumberFormat.getInstance(l).parse(s, pos);
13          boolean all = pos.getIndex() == s.length();          // everything used?
14          System.out.printf("%-9s %-6s -> %s%n", s, l, all ? n : "error at " + pos.getIndex
            ());
15      }
16      static void date(String s, Locale l) {
17          DateTimeFormatter f = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).
            withLocale(l);
18          try { System.out.printf("%-10s %-6s -> %s%n", s, l, LocalDate.parse(s, f)); }
19          catch (DateTimeParseException e) { System.out.printf("%-10s %-6s -> %s%n", s, l, "
            not a date"); }
20      }
21      public static void main(String[] args) {
22          Locale tr = Locale.of("tr", "TR");
23          number("1.234,5", tr);  number("1.234,5", Locale.US);  number("12,5 kg", tr);
24          date("15.10.2026", tr); date("10/15/26", Locale.US);   date("15.10.2026", Locale.
            US);
25      }
26  }
```

`parse(s, pos)` tells how far it got: `12,5 kg` is not a number, although plain `parse` would return `12.5`. Dates in a short style are ambiguous: `10/15/26` only makes sense in the US.

Parsing numbers and dates safely

codeoutput/ParseInput.java — Java

```
10      static void number(String s, Locale l) {
11          ParsePosition pos = new ParsePosition(0);
12          Number n = NumberFormat.getInstance(l).parse(s, pos);
13          boolean all = pos.getIndex() == s.length();          // everything used?
14          System.out.printf("%-9s %-6s -> %s%n", s, l, all ? n : "error at " + pos.getIndex
            ());
15      }
16      static void date(String s, Locale l) {
17          DateTimeFormatter f = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).
            withLocale(l);
18          try { System.out.printf("%-10s %-6s -> %s%n", s, l, LocalDate.parse(s, f)); }
19          catch (DateTimeParseException e) { System.out.printf("%-10s %-6s -> %s%n", s, l, "
            not a date"); }
20      }
21      public static void main(String[] args) {
22          Locale tr = Locale.of("tr", "TR");
23          number("1.234,5", tr);  number("1.234,5", Locale.US);  number("12,5 kg", tr);
24          date("15.10.2026", tr); date("10/15/26", Locale.US);   date("15.10.2026", Locale.
            US);
25      }
26  }
```

Terminal
codeoutput/ParseInput.txt

```
1.234,5   tr_TR  -> 1234.5
1.234,5   en_US  -> error at 5
12,5 kg   tr_TR  -> error at 4
15.10.2026 tr_TR  -> 2026-10-15
10/15/26   en_US  -> 2026-10-15
15.10.2026 en_US  -> not a date
```

`parse(s, pos)` tells how far it got: `12,5 kg` is not a number, although plain `parse` would return `12.5`. Dates in a short style are ambiguous: `10/15/26` only makes sense in the US.

Comparing Text

Collator strength

codeoutput/Strength.java — Java

```java
import java.text.Collator;
import java.util.Locale;

public class Strength {
    public static void main(String[] args) {
        Collator c = Collator.getInstance(Locale.of("tr"));
        String[][] pairs = {{"ilaç", "İlaç"}, {"cam", "çam"
        }, {"ilik", "ılık"}, {"Ankara", "ankara"}};
        int[] levels = {Collator.PRIMARY, Collator.
        SECONDARY, Collator.TERTIARY};
        String[] names = {"PRIMARY", "SECONDARY", "TERTIARY
        "};
        for (int i = 0; i < levels.length; i++) {
            c.setStrength(levels[i]);
            StringBuilder line = new StringBuilder(String.
        format("%-10s", names[i]));
            for (String[] p : pairs)
                line.append(String.format("%s=%s:%-6s ", p
        [0], p[1], c.equals(p[0], p[1])));
            System.out.println(line);
        }
    }
}
```

PRIMARY: only base letters (c = ç is false in Turkish: different letters!). SECONDARY: accents count. TERTIARY: case counts too. Use PRIMARY for search, TERTIARY for sorting.

Collator strength

codeoutput/Strength.java — Java

```java
import java.text.Collator;
import java.util.Locale;

public class Strength {
    public static void main(String[] args) {
        Collator c = Collator.getInstance(Locale.of("tr"));
        String[][] pairs = {{"ilaç", "İlaç"}, {"cam", "çam"
        }, {"ilik", "ılık"}, {"Ankara", "ankara"}};
        int[] levels = {Collator.PRIMARY, Collator.
        SECONDARY, Collator.TERTIARY};
        String[] names = {"PRIMARY", "SECONDARY", "TERTIARY
        "};
        for (int i = 0; i < levels.length; i++) {
            c.setStrength(levels[i]);
            StringBuilder line = new StringBuilder(String.
        format("%-10s", names[i]));
            for (String[] p : pairs)
                line.append(String.format("%s=%s:%-6s ", p
        [0], p[1], c.equals(p[0], p[1])));
            System.out.println(line);
        }
    }
}
```

Terminal — codeoutput/Strength.txt

```
PRIMARY   ilaç=İlaç:true   cam=çam:false  ilik=ılık:false  Ankara=ankara
          :true
SECONDARY ilaç=İlaç:true   cam=çam:false  ilik=ılık:false  Ankara=ankara
          :true
TERTIARY  ilaç=İlaç:false  cam=çam:false  ilik=ılık:false  Ankara=ankara
          :false
```

PRIMARY: only base letters (c = ç is false in Turkish: different letters!). SECONDARY: accents count. TERTIARY: case counts too. Use PRIMARY for search, TERTIARY for sorting.

Testing in Many Locales

One loop, seven locales

Terminal codeoutput/AllLocales.txt

```
$ for tag in en-US en-GB tr-TR de-DE de-CH fr-FR pt-BR; do echo "== $tag"; java -cp out edu.yeditepe.shop.Main $tag | tail -2; done
== en-US
Total: TRY3,749.70
Delivery on October 20, 2026
== en-GB
Total: TRY3,749.70
Delivery on 20 October 2026
== tr-TR
Toplam: ₺3.749,70
20 Ekim 2026 tarihinde teslim
== de-DE
Summe: 3.749,70 TRY
Lieferung am 20. Oktober 2026
== de-CH
Summe: TRY 3’749.70
Lieferung am 20. Oktober 2026
== fr-FR
Total: 3 749,70 TRY
Delivery on 20 octobre 2026
== pt-BR
Total: TRY 3.749,70
Delivery on 20 de outubro de 2026
```

Run every test in several locales: at least English, Turkish (case and comma) and German or French (long words, spaces). `de-CH` and `pt-BR` have no own file: they reuse `_de` and the base file.

Pseudo-localization: a small tool

codeoutput/Pseudo/Pseudo.java — Java

```
 8 public class Pseudo {
 9     static final Map<Character, Character> ACCENT = Map.of(
10         'a', 'à', 'e', 'é', 'i', 'ï', 'o', 'ö', 'u', 'ü', 'A', 'Å', 'E', 'É', 'O', 'Ø');
11
12     static String pseudo(String text) {
13         StringBuilder b = new StringBuilder("[");
14         int depth = 0;                                  // inside {...}: keep as it is
15         for (char c : text.toCharArray()) {
16             if (c == '{') depth++;
17             if (c == '}') depth--;
18             b.append(depth == 0 ? ACCENT.getOrDefault(c, c) : c);
19         }
20         return b.append(" ").append("~".repeat(text.length() / 3)).append("]").toString();
21     }
22     public static void main(String[] args) throws IOException {
```

A fake language that everyone can still read. Lines 23–30 (not shown) apply `pseudo` to every value of `Messages.properties` and write `Messages_en_XA.properties`.

Pseudo-localization: the result

Terminal codeoutput/Pseudo.txt

```
$ java Pseudo.java out/edu/yeditepe/shop/i18n
$ grep -v "^#" out/edu/yeditepe/shop/i18n/Messages_en_XA.properties
title=[Ørdér sümmàry ~~~~]
items=[{0,choice,0#no items|1#one item|1<{0,number,integer} items} ~~~~~~~~~~~~~~~~~~]
total=[Tötàl: {0} ~~~]
delivery=[Délïvéry ön {0} ~~~~~]
$ java -cp out edu.yeditepe.shop.Main en-XA
[Ørdér sümmàry ~~~~]
[3 items ~~~~~~~~~~~~~~~~~~]
[Tötàl: TRY3,749.70 ~~~]
[Délïvéry ön October 20, 2026 ~~~~~]
```

Hard-coded texts would stay plain (easy to spot); accents test fonts and encodings; ~~ makes texts longer to test the layout. `{...}` is kept, so values still work (texts inside `choice` stay plain).

With Maven or Gradle

```
src/main/java/edu/yeditepe/shop/Main.java
src/main/resources/edu/yeditepe/shop/i18n/
    Messages.properties
    Messages_tr.properties
    Messages_de.properties
src/test/java/...      # tests in several locales
```

- Resources are copied to the class path and packed into the JAR automatically.
- Set the encoding once:
  `<project.build.sourceEncoding>UTF-8</...>`
- Run the tests with another locale:
  `mvn test`
  `-DargLine="-Duser.language=tr"`

Not run here (Maven needs the internet).

Summary

Recipes

goal	how
texts of a package	`getBundle("edu.yeditepe.shop.i18n.Messages", l)`
texts with values	`new MessageFormat(rb.getString(key), l).format(values)`
plurals	`{0,choice,0#...|1#...|1<{0,number,integer} ...}`
money	`getCurrencyInstance(l)` + `setCurrency(...)`, `BigDecimal`
dates	`ofLocalizedDate(FormatStyle.LONG).withLocale(l)`, then pass the `String`
no default-locale fallback	`Control.getNoFallbackControl(Control.FORMAT_DEFAULT)`
safe parsing	`parse(s, pos)` and check `pos.getIndex() == s.length()`
search vs. sort	`Collator` strength PRIMARY vs. TERTIARY
bundle encoding	UTF-8 (Java 9+), or `\uXXXX`
find missing texts	pseudo-localization, tests in several locales

Exercises

1. Add `Messages_fr.properties` and run `AllLocales` again.
2. Give `Main` a `-currency EUR` option. What must change in the bundles? (Nothing!)
3. Make the shop use `getNoFallbackControl` and repeat the trap experiment.
4. Translate *"{0} sent {1} files to {2}"* into Turkish and German with numbered placeholders.
5. Write a test that fails when a key exists in `Messages` but not in `Messages_tr`.
6. Use `Collator` with PRIMARY strength to search names in a list without caring about case.

Questions?